

DD2356 VT26 Project Report

Pengyu Wang's Part: MPI Scaling, Optimization, and GPU Offloading

Group 26

1 Design and Implementation

1.1 MPI Decomposition and Communication

The MPI implementation uses a destination-node block decomposition. For a graph with N vertices and P MPI ranks, rank r owns a contiguous vertex interval $[start_r, end_r)$ and computes only the corresponding entries of the next PageRank vector. The graph is stored in the same incoming-edge CSR layout as the serial baseline: `row_ptr[v]` to `row_ptr[v+1]-1` enumerates all source vertices contributing to destination vertex v . To keep the first distributed-memory version simple and easy to verify, each rank stores the full CSR graph and the full current PageRank vector, while only the newly computed block `pr_new[start:end]` is rank-local.

The graph is loaded on rank 0 and distributed to the other ranks with `MPI_Bcast`. Each PageRank iteration then performs the following communication:

1. One scalar `MPI_Allreduce` for the global dangling-node mass.
2. One scalar `MPI_Allreduce` for the global L_1 convergence difference.
3. One `MPI_Allgatherv` to assemble the full next PageRank vector on every rank.

The per-iteration cost can therefore be modeled as

$$T_{\text{iter}}(P) \approx T_{\text{comp}} \left(\frac{M}{P} \right) + 2T_{\text{allreduce}}(P) + T_{\text{allgatherv}}(N, P),$$

where M is the number of edges. This model predicts that the implementation will be communication-bound for small graphs or when the full-vector `Allgatherv` dominates the reduced local sparse update.

1.2 Hybrid MPI+OpenMP

The hybrid version keeps the same MPI node-block decomposition and adds OpenMP parallelism inside each MPI rank. The dangling-node mass and convergence checks use OpenMP reductions, while the sparse incoming-edge PageRank update uses `#pragma omp parallel for schedule(dynamic, 256)`. The dynamic schedule was chosen because real graphs such as `polblogs.csv` have highly irregular in-degree distributions; assigning a fixed number of destination vertices to each thread can leave some threads with many more incoming edges than others.

For comparison with the GPU-offloading path, the group hybrid implementation supports fixed-core experiments with configurations `1x16`, `2x8`, `4x4`, `8x2`, and `16x1`, where the first number is the MPI rank count P and the second number is the OpenMP thread count N per rank. These runs are intended to identify whether the best performance comes from fewer MPI ranks with more threads, or more MPI ranks with less intra-rank threading.

1.3 OpenMP GPU Offloading

The GPU comparison path is implemented with OpenMP target offloading and has two variants. The naive baseline remaps graph and PageRank data for target kernels, while the persistent version establishes an outer device-data region, retains the CSR arrays and ping-pong PageRank buffers on the device across all iterations, and performs dangling-mass and convergence reductions on the target. Formal runs set `PR_REQUIRE_DEVICE=1`, so CPU fallback causes a failed run rather than an accepted measurement. Since the course allocation did not provide a Dardel GPU job, the confirmed-device measurement used the DD2356 Small GPU server: an NVIDIA H100 MIG 1g.10gb allocation. A Hybrid 4x2 control was run on the CPU resources of that same server.

2 Correctness Evaluation

All parallel outputs were compared against the serial PageRank reference using the repository verifier and a tolerance of 10^{-6} . The MPI implementation passed the full nine-dataset correctness suite on Dardel for $P = 1, 2, 4$ ranks, and the formal Dardel scaling experiments passed for all measured $P = 1, 2, 4, 8, 16$ cases in a two-node main-partition job. On the school cluster, the MPI strong-scaling runs for both `polblogs.csv` and `synthetic_100k_1m.csv`, as well as the weak-scaling sequence up to the 200k-node/2M-edge graph, passed for all measured rank counts. The hybrid fixed-core runs passed for every tested (P, N) combination on both datasets. The maximum errors are at machine precision scale, e.g. at most 1.04×10^{-17} in the Dardel `polblogs.csv` MPI run. The formal GPU run additionally passed all nine course datasets with `executed_on_device=YES`; both timed GPU variants passed on the synthetic comparison graph with maximum error 1.02×10^{-20} .

Table 1: Correctness summary for Wang-owned implementations.

Implementation	Dataset scope	Configuration	Result
MPI	9 course datasets	1, 2, 4 ranks on Dardel	PASS, tol. 10^{-6}
MPI	Dardel scaling datasets	1–16 ranks, two main nodes	PASS, tol. 10^{-6}
MPI	2 cluster datasets	1, 2, 4, 8, 16 ranks on school cluster	PASS, tol. 10^{-6}
MPI	cluster weak-scaling sequence	1 to 16 ranks; up to 200k nodes / 2M edges	PASS, tol. 10^{-6}
Hybrid MPI+OpenMP	2 cluster datasets	fixed 16 cores: 1x16 to 16x1	PASS, tol. 10^{-6}
OpenMP target GPU	9 course datasets	H100 MIG persistent target path; device verified	PASS vs. serial

3 Scaling Evaluation

3.1 Dardel MPI Strong Scaling

The formal Dardel run was submitted as Slurm job 20957663 on the main partition using compute nodes `nid001120` and `nid001121`. A placement probe outside each measured interval records that every $P = 2, 4, 8, 16$ case is balanced across both nodes; the $P = 1$ row is the one-node reference. Table 2 reports the strong-scaling result for the course `polblogs.csv` graph. The graph is small (1224 vertices and 19090 edges), so the communication overhead is much larger than the useful local sparse update once more than one rank is used.

Table 2: Two-node MPI strong scaling on Dardel for `polblogs.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.002364	0.0463	1.000000	1.000000	1.000
2	5	0.003274	0.9144	0.722008	0.361004	1.887
4	5	0.003406	0.9637	0.694110	0.173528	3.129
8	5	0.003346	0.9805	0.706557	0.088320	4.412
16	5	0.003732	0.9896	0.633474	0.039592	4.536

To separate graph-size effects from implementation effects, the same experiment was repeated on a larger synthetic graph with 100000 vertices and 1000000 edges. The larger graph has much better block balance (max/average in-edge imbalance around 1.05). Table 3 shows useful scaling through $P = 8$, where speedup reaches $2.98\times$, but the speedup declines at $P = 16$ as collective communication grows.

Table 3: Two-node MPI strong scaling on Dardel for `synthetic_100k_1m.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.036330	0.0395	1.000000	1.000000	1.000
2	5	0.020386	0.1462	1.782080	0.891040	1.035
4	5	0.012707	0.3017	2.859025	0.714756	1.044
8	5	0.012205	0.5257	2.976714	0.372089	1.047
16	5	0.014107	0.7600	2.575258	0.160954	1.051

For the synthetic graph, communication fraction increases from 0.1462 at $P = 2$ to 0.7600 at $P = 16$. The work imbalance remains only 1.051 at $P = 16$, whereas `MPI_Allgatherv` alone grows to an average of 0.008713 s. Thus the two-node result demonstrates useful parallelism for a larger graph, but also shows that replicated-vector collectives limit high-rank efficiency.

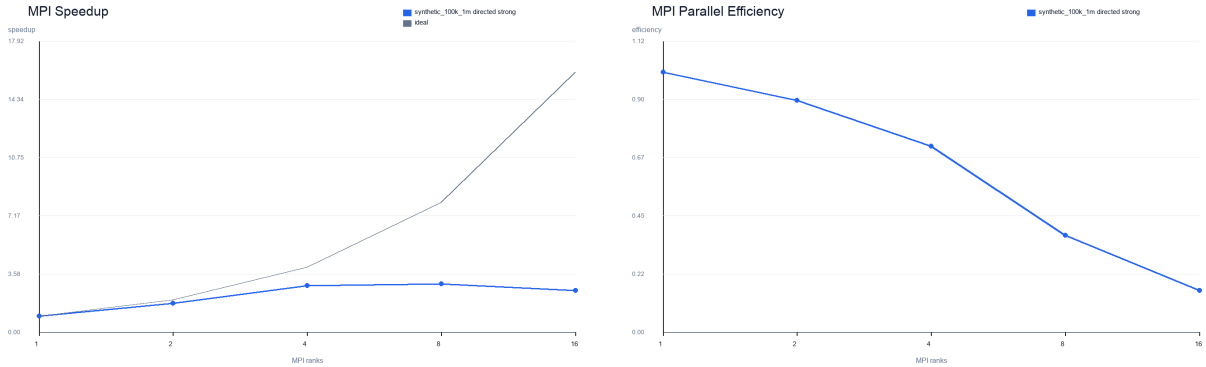


Figure 1: Dardel two-node MPI speedup and parallel efficiency for the synthetic graph.

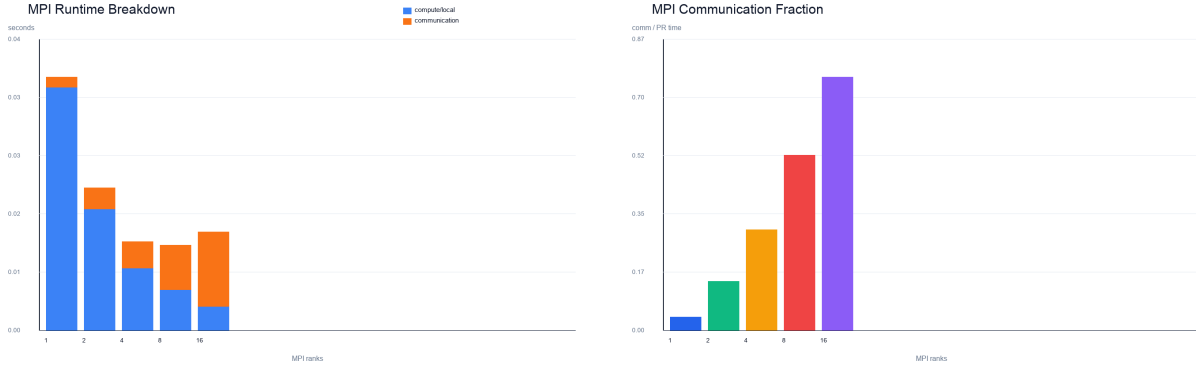


Figure 2: Dardel two-node MPI runtime breakdown and communication fraction.



Figure 3: Dardel two-node MPI workload imbalance measured as max in-edges divided by average in-edges.

3.2 Dardel MPI Weak Scaling

Weak scaling keeps the per-rank graph size approximately fixed while increasing the number of ranks. For this project I used 12500 vertices and 125000 edges per rank, so the required 16-rank experiment uses a 200000-vertex, 2000000-edge synthetic graph. This is a different experiment from strong scaling: strong scaling fixes the graph size, while weak scaling grows the global graph so each rank receives similar local work. The same two-node placement policy as in strong scaling is used for all multi-rank rows.

Table 4: Two-node MPI weak scaling on Dardel with 125k edges per rank.

Dataset	Ranks	Nodes	Edges	PR time (s)	Comm frac.	Weak eff.	Status
weak_1rank_12500_125000	1	12500	125000	0.003778	0.0353	1.000000000	PASS
weak_2rank_25000_250000	2	25000	250000	0.005837	0.3138	0.647216036	PASS
weak_4rank_50000_500000	4	50000	500000	0.006134	0.3398	0.615898790	PASS
weak_8rank_100000_1000000	8	100000	1000000	0.011932	0.5365	0.316600181	PASS
weak_16rank_200000_2000000	16	200000	2000000	0.044137	0.5960	0.085592587	PASS

The 16-rank row satisfies the required weak-scaling case and confirms that the dynamic graph reader can process the larger 200k-node input. However, weak efficiency drops sharply because the replicated-vector algorithm still communicates the full global PageRank vector every iteration. In the 16-rank weak-scaling run, MPI_Allgatherv alone averages 0.020449 s out of 0.044137 s PageRank time, so increasing local work per rank is not enough to hide the global communication cost.

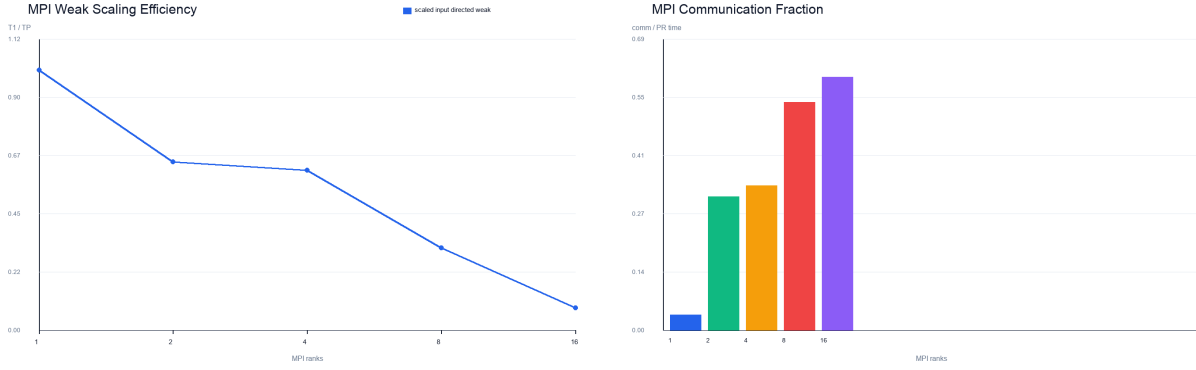


Figure 4: Dardel two-node weak-scaling efficiency and communication fraction.

3.3 School-Cluster MPI Strong Scaling

The same MPI profiler was run on the school cluster. Table 5 shows that the small `polblogs.csv` graph remains communication-bound: the one-rank average PR time is only 0.001484 s, and adding ranks increases the communication fraction to 0.84–0.99. This is the expected regime where collective synchronization dominates useful work.

Table 5: MPI strong scaling on the school cluster for `polblogs.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.001484	0.0142	1.000000	1.000000	1.000
2	5	0.001889	0.8397	0.785472	0.392736	1.887
4	5	0.002927	0.8878	0.506936	0.126734	3.129
8	5	0.002461	0.9801	0.602844	0.075356	4.412
16	5	0.002844	0.9898	0.521733	0.032608	4.536

For the larger synthetic graph, the school-cluster MPI run scales up to four ranks before communication dominates. Table 6 shows the best average speedup at $P = 4$ with $2.14\times$ speedup and 0.534 parallel efficiency. From $P = 8$ onward, the communication fraction rises from 0.44 to 0.75 and then 0.85, so the extra ranks no longer compensate for the collective communication cost. The raw rank-16 measurements include one large outlier (PR time = 0.070486 s); therefore the table keeps the summary average as the main metric while the median rank-16 time (0.012609 s) is useful as a robustness check.

Table 6: MPI strong scaling on the school cluster for `synthetic_100k_1m.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.027228	0.0137	1.000000	1.000000	1.000
2	5	0.017646	0.1766	1.543006	0.771503	1.035
4	5	0.012738	0.4356	2.137590	0.534398	1.044
8	5	0.015967	0.7505	1.705301	0.213163	1.047
16	5	0.023347	0.8478	1.166240	0.072890	1.051

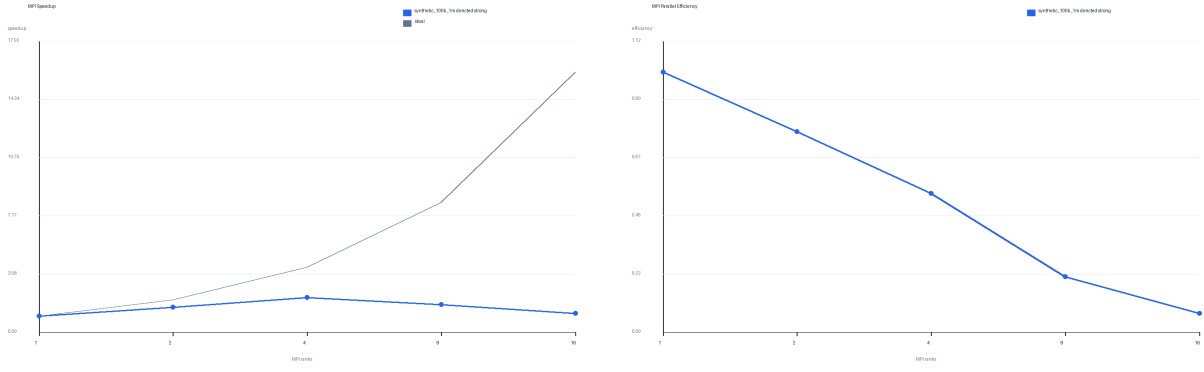


Figure 5: School-cluster MPI speedup and parallel efficiency for `synthetic_100k_1m.csv`.

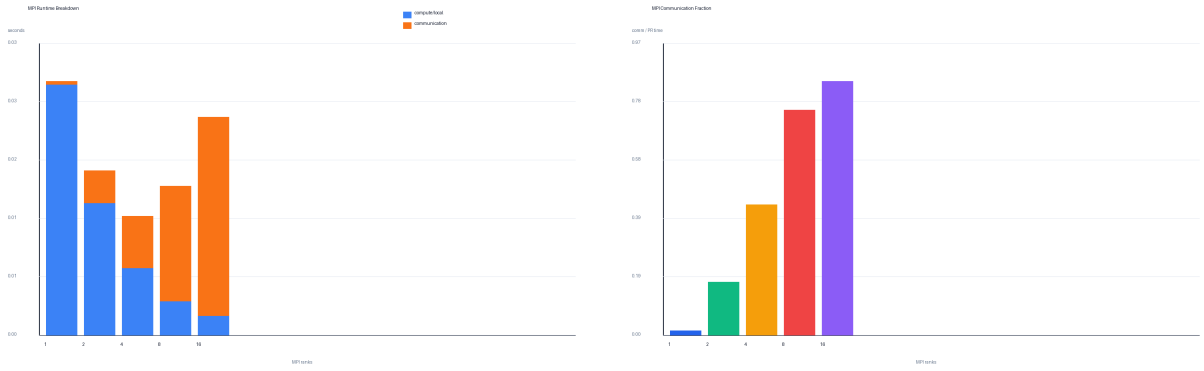


Figure 6: School-cluster MPI runtime breakdown and communication fraction.

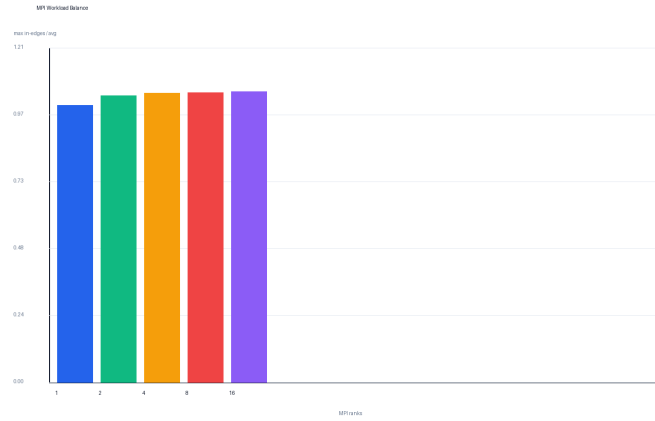


Figure 7: School-cluster MPI workload imbalance for `synthetic_100k_1m.csv`.

3.4 School-Cluster MPI Weak Scaling

The school-cluster weak-scaling experiment used the same generated graph instances as the Dardel experiment, with 125000 edges assigned per rank and a 16-rank maximum input of 200000 vertices and 2000000 edges. Equality of the per-rank in-edge distributions in the two result CSV files confirms that the comparisons are made on identical graph instances.

Table 7: MPI weak scaling on the school cluster with 125k edges per rank.

Dataset	Ranks	Nodes	Edges	PR time (s)	Comm frac.	Weak eff.	Status
weak_1rank_12500_125000	1	12500	125000	0.003310	0.0123	1.000000000	PASS
weak_2rank_25000_250000	2	25000	250000	0.005345	0.2442	0.619298836	PASS
weak_4rank_50000_500000	4	50000	500000	0.007542	0.4821	0.438951947	PASS
weak_8rank_100000_1000000	8	100000	1000000	0.011587	0.5887	0.285694560	PASS
weak_16rank_200000_2000000	16	200000	2000000	0.024371	0.6814	0.135833573	PASS

Although the school cluster performs substantially better than the measured Dardel job, its weak efficiency still falls from 1.0 to 0.1358 by 16 ranks. The per-rank incoming-edge imbalance remains small (at most 1.051), while communication consumes 68.14% of PageRank time at 16 ranks. Therefore, the weak-scaling deterioration is attributable primarily to the replicated-vector collectives, rather than to uneven sparse work assignment.

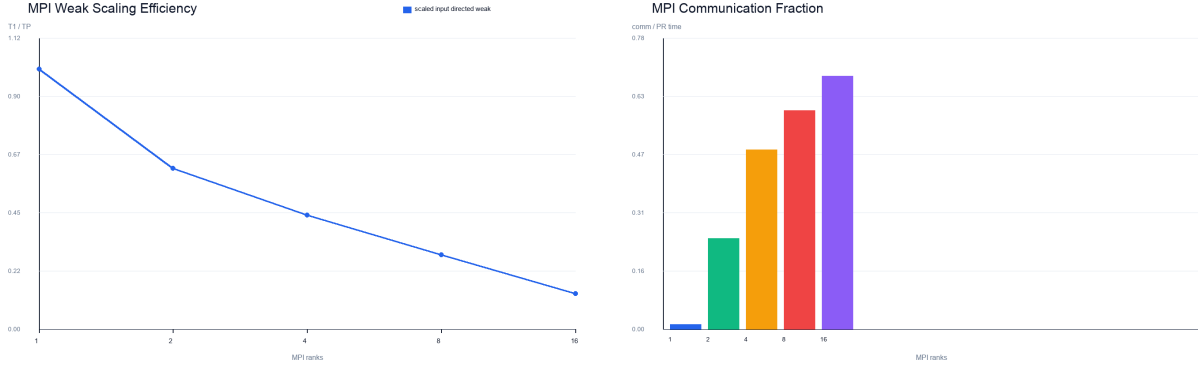


Figure 8: School-cluster weak-scaling efficiency and communication fraction.

4 Optimization Effectiveness

The first optimization is to precompute `inv_out_degree[u]` during graph construction. The PageRank inner loop then uses multiplication instead of performing a floating-point division for every incoming edge contribution:

$$s_v = \sum_{u \in \text{In}(v)} pr_u \cdot \text{inv_out_degree}[u].$$

This directly targets the sparse-update kernel, which is the dominant computation in the serial and single-rank paths.

The second optimization is the hybrid sparse-loop schedule. Because destination vertices can have very different numbers of incoming edges, `schedule(dynamic, 256)` reduces thread-level imbalance compared with a pure static assignment. This does not remove MPI collective overhead, but it improves the local work distribution within each rank for irregular graphs.

To verify these choices directly, I added an ablation profiler that builds four hybrid variants: `no_inv_static`, `inv_static`, `no_inv_dynamic`, and `inv_dynamic`. The formal run was conducted on the DD2356 school-cluster CPU server with OpenMPI core binding enabled. For each variant and layout, one warmup preceded ten correctness-checked timing repetitions. The measured layouts were 1x1, 1x2, 1x4, 1x8, 1x16, 2x8, 4x4, 8x2, and 16x1.

The profiler now measures both total PageRank time and sparse update-kernel time. A diagnostic traversal outside the timed interval also counts the incoming edges processed by each OpenMP worker; its imbalance metric is $\max_t M_t / \overline{M}_t$. Thus this metric observes thread work directly rather than inferring thread behavior from MPI rank imbalance. Parallel efficiency is computed separately for each variant from its own single-worker baseline:

$$S_v(P, N) = T_v(1, 1) / T_v(P, N),$$

$$E_v(P, N) = S_v(P, N) / (PN).$$

For irregular-load evidence, a controlled `skewed_100k_1m.csv` input first inserts one cycle edge per node and directs 80% of remaining edges to the first 6250 destination nodes. This preserves a deterministic, contiguous hot region for comparing static and dynamic schedules. Both formal summary CSVs contain all $4 \times 9 = 36$ rows, the raw CSVs contain 360 timed samples each, and every sample passes verification against the serial reference.

Table 8: Formal optimization evidence on the regular synthetic graph. PR and update times are seconds; efficiency is relative to each variant’s own 1x1 baseline.

Config	PR before	PR full opt.	Speedup	Update before	Update full opt.	E before	E full opt.
1x1	0.035842	0.032455	1.1043	0.033078	0.029548	1.0000	1.0000
1x4	0.012826	0.010960	1.1702	0.010691	0.008779	0.6986	0.7403
1x8	0.006531	0.006945	0.9403	0.004768	0.004852	0.6860	0.5841
1x16	0.005576	0.005765	0.9672	0.003683	0.002765	0.4017	0.3518
2x8	0.006187	0.005274	1.1732	0.003532	0.002912	0.3621	0.3846
4x4	0.007609	0.007524	1.0113	0.003938	0.003026	0.2944	0.2696

Table 9: Dynamic-scheduling evidence on the controlled skewed graph. The static column uses `inv_static`; the dynamic column uses `inv_dynamic`.

Config	Static imbalance	Dynamic imbalance	Reduction	Static update (s)	Dynamic update (s)
1x4	3.1591	1.1489	2.750×	0.028158	0.010108
1x8	6.0388	1.2420	4.862×	0.020228	0.005293
1x16	11.7988	1.2415	9.504×	0.017573	0.003117

The formal results show two different regimes. On the regular synthetic graph, the complete optimization improves PageRank time by $1.1043\times$ at 1x1, $1.1702\times$ at 1x4, and $1.1732\times$ at 2x8; the efficiency gain at 1x4 is $+0.0417$. It is not a universal scaling improvement: at 1x16, update-kernel time falls from 0.003683 to 0.002765 s while end-to-end PageRank time is slightly worse, and efficiency falls by 0.0499. Synchronization and scheduling overhead therefore offsets the kernel reduction at that layout. On the controlled skewed graph, dynamic scheduling directly addresses the second bottleneck: at 1x16, imbalance falls from 11.7988 to 1.2415 and update-kernel time falls from 0.017573 to 0.003117 s. This is a measured thread-load result, not a claim inferred from MPI-rank work.

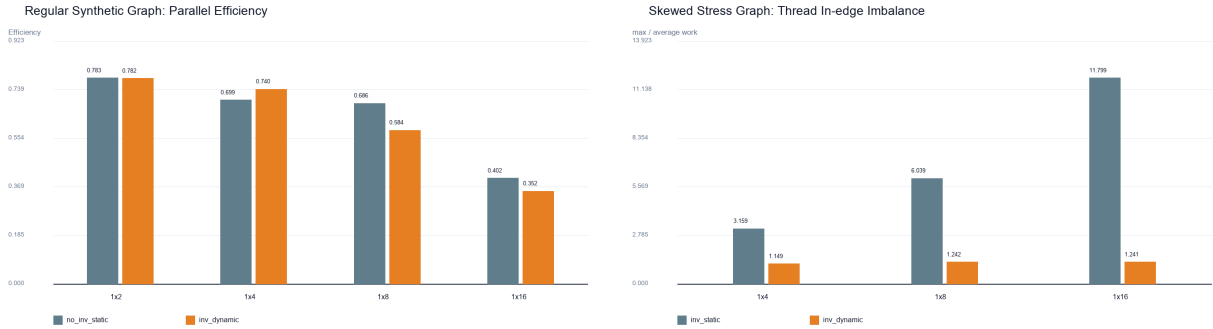


Figure 9: Before/after parallel efficiency on the regular graph (left), and static versus dynamic thread imbalance on the skewed stress graph (right).

OpenMP target execution was formally confirmed on the DD2356 Small GPU server: the run detected one NVIDIA H100 MIG 1g.10gb target device, and `omp_is_initial_device()` confirmed that target regions executed on device. On the directed synthetic_100k_1m.csv graph, retaining arrays across PageRank iterations reduces average PR time from 0.059829 s for the naive path to 0.033684 s for the persistent path, an improvement of $1.7762\times$. Persistent offload is also slightly faster than the serial reference for this kernel metric ($1.0822\times$). However, the same-server Hybrid 4x2 control completes PR computation in 0.010364 s, making it

3.2500 $\times$ faster than persistent GPU offloading on this one-million-edge input. Thus the data show a successful GPU optimization and a fair comparison, but not a GPU win over the CPU hybrid implementation at the tested scale.

Table 10: Confirmed-device GPU comparison on the DD2356 Small GPU server for `synthetic_100k_1m.csv` (directed), five-run averages.

Variant	Configuration	PR time (s)	Correctness
Serial	1 CPU thread	0.036452	reference
OpenMP target GPU	naive mapping	0.059829	PASS, device verified
OpenMP target GPU	persistent mapping	0.033684	PASS, device verified
Hybrid MPI+OpenMP	4x2, 8 CPU workers	0.010364	PASS

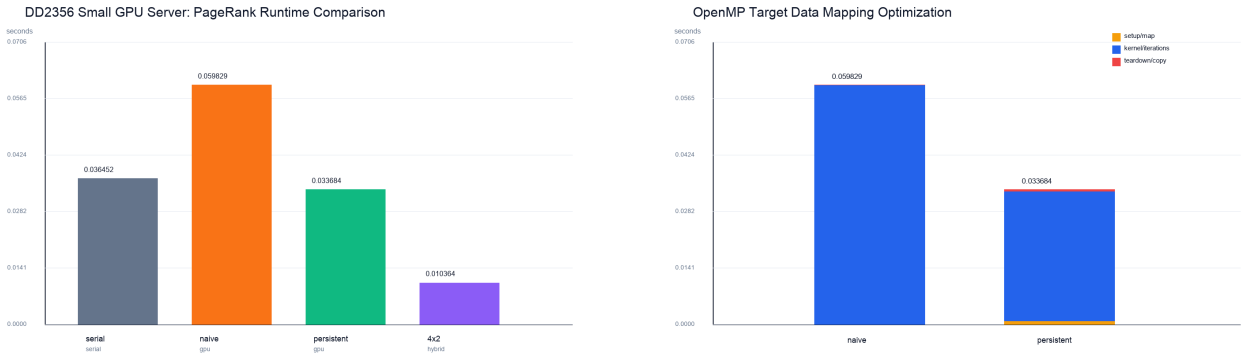


Figure 10: Same-server GPU and Hybrid timing comparison (left), and OpenMP target mapping optimization breakdown (right).

For hybrid MPI+OpenMP, the fixed-core experiments keep the total number of CPU workers at 16 and vary the split between MPI ranks and OpenMP threads. The reported speedup is relative to the first fixed-core configuration 1x16; it is not a speedup over the serial baseline.

Table 11: Hybrid fixed-core sweep on the school cluster for `polblogs.csv` (directed).

Ranks	Threads/rank	PR time (s)	Comm frac.	Speedup vs. 1x16	Status
1	16	0.024310	0.0018	1.0000	PASS
2	8	0.010850	0.1643	2.2406	PASS
4	4	0.005325	0.7428	4.5652	PASS
8	2	0.004281	0.8655	5.6783	PASS
16	1	0.002907	0.9533	8.3631	PASS

Table 12: Hybrid fixed-core sweep on the school cluster for `synthetic_100k_1m.csv` (directed).

Ranks	Threads/rank	PR time (s)	Comm frac.	Speedup vs. 1x16	Status
1	16	0.036692	0.0137	1.0000	PASS
2	8	0.018884	0.1758	1.9431	PASS
4	4	0.011843	0.6741	3.0981	PASS
8	2	0.019373	0.7682	1.8940	PASS
16	1	0.012234	0.8220	2.9992	PASS

The best hybrid split depends on graph size. For `polblogs.csv`, the best fixed-core result is 16x1, which avoids OpenMP overhead and benefits from the very small local work size despite high communication fraction. For `synthetic_100k_1m.csv`, the best average result is 4x4: it exposes enough MPI parallelism while keeping collective communication lower than the 8x2 and 16x1 configurations.

5 A-Level Coverage and Remaining Measurements

The Dardel main-partition job now closes the previously missing multi-node MPI requirement: every multi-rank strong/weak row is placed across `nid001120` and `nid001121`, and the 16-rank weak row uses the required 200k-node/2M-edge graph. Confirmed-device OpenMP target measurements on the DD2356 Small GPU server also close Wang’s GPU-offloading comparison. The formal school-cluster CPU optimization experiment closes Wang’s profiling-guided optimization evidence with update-kernel, thread-imbalance, and parallel-efficiency measurements. The remaining entries below belong to Minyi Zhu’s OpenMP/hybrid analysis scope.

Table 13: A-level evidence status after the current result consolidation.

Requirement	Evidence available	Still required
School-cluster MPI strong/weak scaling	Strong and weak CSVs; weak includes verified 16-rank 200k/2M case	Complete for measured rank sequence
Dardel MPI strong/weak scaling on multiple nodes	Verified strong and weak CSVs for 1–16 ranks on two main nodes; host-name record	Complete for measured rank sequence
Hybrid fixed-core search on school cluster	Verified 1x16 through 16x1 sweeps	Minyi scope: repeat for additional total core budgets
OpenMP speedup on three systems	Correctness and local timing evidence	Minyi scope: provide scaling measurements on three systems
GPU offload comparison with Hybrid	Confirmed H100 MIG execution; nine-dataset PASS; same-server GPU naive/persistent and Hybrid 4x2 comparison	Complete for Wang’s GPU scope
Profiling-guided optimizations	Formal CPU ablation: two inputs, four variants, nine layouts, ten repeats; thread imbalance and efficiency included	Complete for Wang’s optimization scope

Contribution

Pengyu Wang implemented the MPI decomposition and communication timing, ran and analyzed the Dardel multi-node and school-cluster MPI strong/weak-scaling experiments, added the 16-rank weak-scaling workflow with dynamically sized graph readers, integrated the MPI result pipeline, and implemented and measured OpenMP GPU offloading. Wang also completed the formal CPU ablation evidence for reciprocal out-degree caching and dynamic scheduling, including before/after parallel efficiency and controlled thread-imbalance measurements. The final GPU evidence confirms execution on an NVIDIA H100 MIG device, demonstrates the persistent-device-data optimization against the naive target baseline, and compares it with the group Hybrid 4x2 baseline on the same server and graph.